

Технологии и методы программирования

лекция 5

Ищенко Алексей Петрович

Модули

- Модулем называют автономно компилируемую программную единицу. Термин «модуль» традиционно используется в двух смыслах. Первоначально, когда размер программ был сравнительно невелик и все подпрограммы компилировались отдельно, под модулем понималась подпрограмма, т.е. последовательность связанных фрагментов программы, обращение к которой выполняется по имени.
- Со временем, когда размер программ значительно вырос и появилась возможность создавать библиотеки ресурсов: констант, переменных, описаний типов, классов и подпрограмм, термин «модуль» стал использоваться и в смысле автономно компилируемого набора программных ресурсов. Данный модуль может получать и/или возвращать через общие области памяти или параметры.

Модули

Первоначально к модулям (ещё понимаемым как подпрограммы) предъявлялись следующие требования:

- отдельная компиляция;
- одна точка входа;
- одна точка выхода;
- соответствие принципу вертикального управления;
- возможность вызова других модулей;
- небольшой размер (до 50 – 60 операторов языка);
- независимость от истории вызовов;
- выполнение одной функции.

Модули

- Требования одной точки входа, одной точки выхода, независимости от истории вызовов и соответствия принципу вертикального управления были вызваны тем, что в то время из-за серьёзных ограничений на объём оперативной памяти программисты были вынуждены разрабатывать программы с максимально возможной повторяемостью кодов. В результате подпрограммы, имеющие несколько точек входа и выхода, не только были обычным явлением, но и считались высоким классом программирования. Следствием же было то, что программы было очень сложно не только модифицировать, но и понять, а иногда и просто полностью отладить.

Модули

Со временем, когда основные требования структурного подхода стали поддерживаться языками программирования и под модулем стали понимать отдельно компилируемую библиотеку ресурсов, требование независимости модулей стало основным.

Практика показала, что чем выше степень независимости модулей, тем:

- легче разобраться в отдельном модуле и всей программе и, соответственно, тестировать, отлаживать и модифицировать её;
- меньше вероятность появления новых ошибок при исправлении старых или внесении изменений в программу, т.е. вероятность появления «волнового» эффекта;
- проще организовать разработку программного обеспечения группой программистов и легче его сопровождать.

Таким образом, уменьшение зависимости модулей улучшает технологичность проекта. Степень независимости модулей (как подпрограмм, так и библиотек) оценивают двумя критериями: сцеплением и связностью.

Сцепление модулей

Сцепление является мерой взаимозависимости модулей, которая определяет, насколько хорошо модули отделены друг от друга. Модули независимы, если каждый из них не содержит о другом никакой информации. Чем больше информации о других модулях хранит модуль, тем больше он с ними сцеплен.

Различают пять типов сцепления модулей:

- по данным;
- по образцу;
- по управлению;
- по общей области данных;
- по содержимому.

Сцепление модулей

- **Сцепление по данным** предполагает, что модули обмениваются данными, представленными скалярными значениями. При небольшом количестве передаваемых параметров этот тип обеспечивает наилучшие технологические характеристики программного обеспечения.

Сцепление модулей

- **Сцепление по образцу** предполагает, что модули обмениваются данными, объединёнными в структуры. Этот тип также обеспечивает неплохие характеристики, но они хуже, чем у предыдущего типа, так как конкретные передаваемые данные «спрятаны» в структуры и потому уменьшается «прозрачность» связи между модулями. Кроме того, при изменении структуры передаваемых данных необходимо модифицировать все использующие её модули.

Сцепление модулей

- При **сцеплении по управлению** один модуль посылает другому некоторый информационный объект (флаг), предназначенный для управления внутренней логикой модуля. Таким способом часто выполняют настройку режимов работы программного обеспечения. Подобные настройки также снижают наглядность взаимодействия модулей и потому обеспечивают ещё худшие характеристики технологичности разрабатываемого программного обеспечения по сравнению с предыдущими типами связей.

Сцепление модулей

Сцепление по общей области данных предполагает, что модули работают с общей областью данных. Этот тип сцепления считается недопустимым, поскольку:

- программы, использующие данный тип сцепления, очень сложны для понимания при сопровождении программного обеспечения;
- ошибка одного модуля, приводящая к изменению общих данных, может проявиться при выполнении другого модуля, что существенно усложняет локализацию ошибок;
- при ссылке к данным в общей области модули используют конкретные имена, что уменьшает гибкость разрабатываемого программного обеспечения.

Сцепление модулей

Следует иметь в виду, что «подпрограммы с памятью», действия которых зависят от истории вызовов, используют сцепление по общей области, что делает их работу в общем случае непредсказуемой. Именно этот вариант используют статические переменные С и С++.

Сцепление модулей

- В случае **сцепления по содержимому** один модуль содержит обращения к внутренним компонентам другого (передаёт управление внутрь, читает и/или изменяет внутренние данные или сами коды), что полностью противоречит блочно-иерархическому подходу. Отдельный модуль в этом случае уже не является блоком («черным ящиком»): его содержимое должно учитываться в процессе разработки другого модуля. Современные универсальные языки процедурного программирования, например Pascal, данного типа сцепления в явном виде не поддерживают, но для языков низкого уровня, например Ассемблера, такой вид сцепления остаётся возможным.

Характеристики различных типов сцепления по экспертным оценкам

Тип сцепления	Сцепление балл	Устойчивость к ошибкам других модулей	Наглядность (понятность)	Возможность изменения	Вероятность повторного использования
По данным	1	Хорошая	Хорошая	Хорошая	Большая
По образцу	3	Средняя	Хорошая	Средняя	Средняя
По управлению	4	Средняя	Плохая	Плохая	Малая
По общей области	6	Плохая	Плохая	Средняя	Малая
По содержанию	10	Плохая	Плохая	Плохая	Малая

Сцепление модулей

- Допустимыми считают первые три типа сцепления, так как использование остальных приводит к резкому ухудшению технологичности программ.
- Как правило, модули сцепляются между собой несколькими способами. Учитывая это, качество программного обеспечения принято определять по типу сцепления с худшими характеристиками. Так, если использовано сцепление по данным и сцепление по управлению, то определяющим считают сцепление по управлению.

Сцепление модулей

- В некоторых случаях сцепление модулей можно уменьшить, удалив необязательные связи и структурировав необходимые связи. Примером может служить объектно-ориентированное программирование, в котором вместо большого количества параметров метод неявно получает адрес области (структуры), в которой расположены поля объекта, и явно дополнительные параметры. В результате модули оказываются сцепленными по образцу.

Связность модулей

- Связность – мера прочности соединения функциональных и информационных объектов внутри одного модуля. Если сцепление характеризует качество отделения модулей, то связность характеризует степень взаимосвязи элементов, реализуемых одним модулем. Размещение сильно связанных элементов в одном модуле уменьшает межмодульные связи и, соответственно, взаимовлияние модулей. В то же время помещение сильно связанных элементов в разные модули не только усиливает межмодульные связи, но и усложняет понимание их взаимодействия. Объединение слабо связанных элементов также уменьшает технологичность модулей, так как такими элементами сложнее мысленно манипулировать.

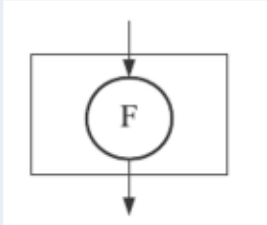
Связность модулей

Различают следующие виды связности (в порядке убывания уровня):

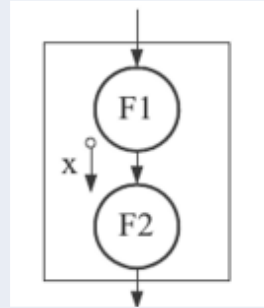
- функциональную;
- последовательную;
- информационную (коммуникативную);
- процедурную;
- временную;
- логическую;
- случайную.

Связность модулей

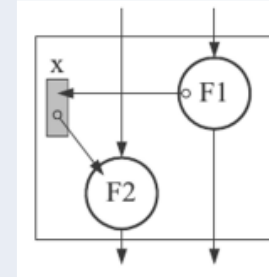
а) функциональная



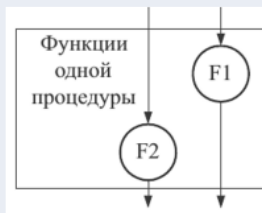
б) последовательная



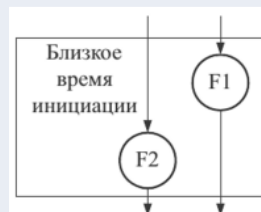
в) информационная



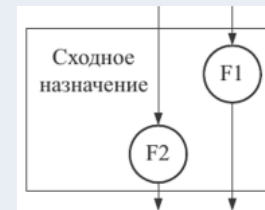
г) процедурная



д) временная



е) логическая



Связность модулей

При **функциональной связности** все объекты модуля предназначены для выполнения одной функции: операции, объединяемые для выполнения одной функции, или данные, связанные с одной функцией. Модуль, элементы которого связаны функционально, имеет чётко определённую цель, при его вызове выполняется одна задача, например подпрограмма поиска минимального элемента массива. Такой модуль имеет максимальную связность, следствием которой являются его хорошие технологические качества: простота тестирования, модификации и сопровождения. Именно с этим связано одно из требований структурной декомпозиции «один модуль – одна связь между модулями-библиотеками ресурсов». Например, если при проектировании текстового редактора предполагается функция редактирования, то лучше организовать модуль библиотеку функций редактирования, чем поместить часть функций в один модуль, а часть – в другой.

Связность модулей

- При **последовательной связности** функций выход одной функции служит исходными данными для другой функции. Как правило, такой модуль имеет одну точку входа, т.е. реализует одну подпрограмму, выполняющую две функции. Считают, что данные, используемые последовательными функциями, также связаны последовательно. Модуль с последовательной связностью функций можно разбить на два или более модулей как с последовательной, так и с функциональной связностью. Такой модуль выполняет несколько функций, и, следовательно, его технологичность хуже: сложнее организовать тестирование, а при выполнении модификации мысленно приходится разделять функции модуля.

Связность модулей

- **Информационно связанными** считают функции, обрабатывающие одни и те же данные. При использовании структурных языков программирования раздельное выполнение функций можно осуществить, только если каждая функция реализуется своей подпрограммой.
- Несмотря на объединение нескольких функций, информационно связанный модуль имеет неплохие показатели технологичности. Это объясняется тем, что все функции, работающие с некоторыми данными, собраны в одно место, что позволяет при изменении формата данных корректировать лишь один модуль. Информационно связанными также считают данные, которые обрабатываются одной функцией.

Связность модулей

- **Процедурно связаны** функции или данные, которые являются частями одного процесса. Обычно модули с процедурной связностью функций получают, если в модуле объединены функции альтернативных частей программы. При процедурной связности отдельные элементы модуля связаны крайне слабо, так как реализуемые ими действия связаны лишь общим процессом, следовательно, технологичность данного вида связи ниже, чем предыдущего.

Связность модулей

- **Временная связность** функций подразумевает, что эти функции выполняются параллельно или в течение некоторого периода времени. Временная связность данных означает, что они используются в некотором временном интервале. Например, временную связность имеют функции, выполняемые при инициализации некоторого процесса. Отличительной особенностью временной связности является то, что действия, реализуемые такими функциями, обычно могут выполняться в любом порядке. Содержание модуля с временной связностью функций имеет тенденцию меняться: в него могут включаться новые действия и/или исключаться старые. Большая вероятность модификации функции ещё больше уменьшает показатели технологичности модулей данного вида по сравнению с предыдущим.

Связность модулей

- Логическая связь базируется на объединении данных или функций в одну логическую группу. В качестве примера можно привести функции обработки текстовой информации или данные одного и того же типа. Модуль с логической связностью функций часто реализует альтернативные варианты одной операции, например сложение целых чисел и сложение вещественных чисел. Из такого модуля всегда будет вызываться одна какая-либо его часть, при этом вызывающий и вызываемый модули будут связаны по управлению. Понять логику работы модулей, содержащих логически связанные компоненты, как правило, сложнее, чем модулей, использующих временную связность, следовательно, их показатели технологичности ещё ниже.

Связность модулей

- В том случае, если связь между элементами мала или отсутствует, считают, что они имеют случайную связность. Модуль, элементы которого связаны случайно, имеет самые низкие показатели технологичности, так как элементы, объединённые в нём, вообще не связаны.
- В трёх предпоследних случаях связь между несколькими подпрограммами в модуле обусловлена внешними причинами, а в последнем – вообще отсутствует. Это соответствующим образом проецируется на технологические характеристики модулей.

Характеристики различных видов связности по экспертным оценкам

Вид связности	Сцепление, балл	Наглядность (понятность)	Возможность изменения	Сопровождается
Функциональная	10	Хорошая	Хорошая	Хорошая
Последовательная	9	Хорошая	Хорошая	Хорошая
Информационная	8	Средняя	Средняя	Средняя
Процедурная	5	Средняя	Средняя	Плохая
Временная	3	Средняя	Средняя	Плохая
Логическая	1	Плохая	Плохая	Плохая
Случайная	0	Плохая	Плохая	Плохая

Связность модулей

- Анализ данных таблицы показывает, что на практике целесообразно использовать функциональную, последовательную и информационную связности.
- Как правило, при хорошо продуманной декомпозиции модули верхних уровней иерархии имеют функциональную или последовательную связность функций и данных.
- Для модулей обслуживания данных характерна информационная связность функций. Данные таких модулей могут быть связаны по-разному. Так, модули, содержащие описание классов при объектно-ориентированном подходе, характеризуются информационной связностью методов и функциональной связностью данных.
- Получение в процессе декомпозиции модулей с другими видами связности, скорее всего, означает недостаточно продуманное проектирование. Исключением являются лишь библиотеки ресурсов.

Библиотеки ресурсов

- Различают библиотеки ресурсов двух типов: библиотеки подпрограмм и библиотеки классов.
- **Библиотеки подпрограмм** реализуют функции, близкие по назначению, например библиотека графического вывода информации. Связность подпрограмм между собой в такой библиотеке – логическая, а связность самих подпрограмм функциональная, так как каждая из них обычно реализует одну функцию.
- **Библиотеки классов** реализуют близкие по назначению классы. Связность элементов класса информационная, связность классов между собой может быть функциональной для родственных или ассоциированных классов и логической для остальных.

Библиотеки ресурсов

- В качестве средства улучшения технологических характеристик библиотек ресурсов в настоящее время широко используют разделение тела модуля на интерфейсную часть и область реализации.
- Интерфейсная часть в данном случае содержит совокупность объявлений ресурсов (заголовков подпрограмм, имен переменных, типов, классов и т.п.), которые данная библиотека предоставляет другим модулям. Ресурсы, объявление которых в интерфейсной части отсутствует, извне не доступны. Область реализации содержит тела подпрограмм и, возможно, внутренние ресурсы (подпрограммы, переменные, типы), используемые этими подпрограммами.
- При такой организации любые изменения реализации библиотеки, не затрагивающие её интерфейс, не требуют пересмотра модулей, связанных с библиотекой, что улучшает технологические характеристики модулей-библиотек. Кроме того, подобные библиотеки, как правило, хорошо отлажены и продуманы, так как часто используются разными программами.

Нисходящая и восходящая разработка программного обеспечения

При проектировании, реализации и тестировании компонентов структурной иерархии, полученной при декомпозиции, применяют два подхода:

- восходящий;
- нисходящий.

В литературе встречается ещё один подход, получивший название «расширение ядра». Он предполагает, что в первую очередь проектируют и разрабатывают некоторую основу – ядро программного обеспечения, например структуры данных и процедуры, связанные с ними. В дальнейшем ядро наращивают, комбинируя восходящий и нисходящий методы. На практике данный подход в зависимости от уровня ядра практически сводится либо к нисходящему, либо к восходящему подходу.

Восходящий подход

- При использовании восходящего подхода сначала проектируют и реализуют компоненты нижнего уровня, затем предыдущего и т.д. По мере завершения тестирования и отладки компонентов осуществляют их сборку, причём компоненты нижнего уровня при таком подходе часто помещают в библиотеки компонентов.
- Для тестирования и отладки компонентов проектируют и реализуют специальные тестирующие программы.

Восходящий подход

Подход имеет следующие недостатки:

- увеличение вероятности несогласованности компонентов вследствие неполноты спецификаций;
- наличие издержек на проектирование и реализацию тестирующих программ, которые нельзя преобразовать в компоненты;
- позднее проектирование интерфейса, а соответственно невозможность продемонстрировать его заказчику для уточнения спецификаций и т.д.

Восходящий подход

- Исторически восходящий подход появился раньше, что связано с особенностью мышления программистов, которые в процессе обучения привыкают при написании небольших программ сначала детализировать компоненты нижних уровней (подпрограммы, классы). Это позволяет им лучше осознавать процессы верхних уровней. При промышленном изготовлении программного обеспечения восходящий подход в настоящее время практически не используют.

Нисходящий подход

- Нисходящий подход предполагает, что проектирование и последующая реализация компонентов выполняются «сверху-вниз», т.е. вначале проектируют компоненты верхних уровней иерархии, затем следующих и так далее до самых нижних уровней. В той же последовательности выполняют и реализацию компонентов. При этом в процессе программирования компоненты нижних, ещё не реализованных уровней заменяют специально разработанными отладочными модулями – «заглушками», что позволяет тестировать и отлаживать уже реализованную часть.
- При использовании нисходящего подхода применяют иерархический, операционный и комбинированный методы определения последовательности проектирования и реализации компонентов.

Нисходящий подход

- **Иерархический метод** предполагает выполнение разработки строго по уровням. Исключения допускаются при наличии зависимости по данным, т.е. если обнаруживается, что некоторый модуль использует результаты другого, то его рекомендуется программировать после этого модуля. Основной проблемой данного метода является большое количество достаточно сложных заглушек. Кроме того, при использовании данного метода основная масса модулей разрабатывается и реализуется в конце работы над проектом, что затрудняет распределение человеческих ресурсов.

Нисходящий подход

- **Операционный метод** связывает последовательность выполнения при запуске программы. Применение метода усложняется тем, что порядок выполнения модулей может зависеть от данных. Кроме того, модули вывода результатов, несмотря на то, что они вызываются последними, должны разрабатываться одними из первых, чтобы не проектировать сложную заглушку, обеспечивающую вывод результатов при тестировании. С точки зрения распределения человеческих ресурсов сложным является начало работ, пока не закончены все модули, находящиеся на так называемом критическом пути.

Нисходящий подход

Комбинированный метод учитывает такие факторы, влияющие на последовательность разработки:

- достижимость модуля – наличие всех модулей в цепочке вызова данного модуля;
- зависимость по данным – модули, формирующие некоторые данные, должны создаваться раньше обрабатывающих;
- обеспечение возможности выдачи результатов – модули вывода результатов должны создаваться раньше обрабатывающих;
- готовность вспомогательных модулей – вспомогательные модули, например модули закрытия файлов, завершения программы, должны создаваться раньше обрабатывающих;
- наличие необходимых ресурсов.

Нисходящий подход

- Кроме того, при прочих равных условиях сложные модули должны разрабатываться прежде простых, так как при их проектировании могут выявиться неточности в спецификациях, а чем раньше это произойдет, тем лучше.
- Нисходящий подход допускает нарушение нисходящей последовательности разработки компонентов в специально оговоренных случаях. Так, если некоторый компонент нижнего уровня используется многими компонентами более высоких уровней, то его рекомендуют проектировать и разрабатывать раньше, чем вызывающие его компоненты. И, наконец, в первую очередь проектируют и реализуют компоненты, обеспечивающие обработку правильных данных, оставляя компоненты обработки неправильных данных напоследок.

Нисходящий подход

Нисходящий подход обычно используют и при объектно-ориентированном программировании. В соответствии с рекомендациями подхода вначале проектируют и реализуют пользовательский интерфейс программного обеспечения, затем разрабатывают классы некоторых базовых объектов предметной области, а уже потом, используя эти объекты, проектируют и реализуют остальные компоненты. Нисходящий подход обеспечивает:

- максимально полное определение спецификаций проектируемого компонента и согласованность компонентов между собой;
- раннее определение интерфейса пользователя, демонстрация которого заказчику позволяет уточнить требования к создаваемому программному обеспечению;
- возможность нисходящего тестирования и комплексной отладки.

Спасибо за внимание!

Ищенко Алексей Петрович